

Grade A Plus - Reason Codes

Error and reason code reference for Frontend and QA

When the backend refuses something, it says why with a **reason code**: a short, fixed name such as `FILE_TOO_LARGE`. This document lists every code, what it means in plain words, the exact text the user sees, and what the response looks like, so that the frontend can handle each one and QA can test each one.

There are **63 codes**. **44** are sent to the user in a response. **19** are written only into the activity record and are never sent to the user.

Where this applies. The contract described here is the one on the **staging** environment, updated on 5 October 2026. All paths are under `/api/v1/`.

Contents

1. [Five Rules for Using Reason Codes](#)
2. [What an Error Response Looks Like](#)
3. [Errors Inside a Batch](#)
4. [Retrying a Failed Item](#)
5. [The Codes, One by One](#)
6. [Sign-in Locks: a Slightly Different Body](#)
7. [Errors That Have No Reason Code](#)
8. [Codes Kept Only in the Activity Record](#)
9. [Limits Worth Knowing for Testing](#)
10. [Quick Index of All User-facing Codes](#)

1. Five Rules for Using Reason Codes

1. **Decide with the code, show the message.** Use `reason_code` to decide what the screen does. Show the user the `message` the backend sends. Never make a decision by reading the message text, because wording can change.
2. **Show the advice too.** `remediation` tells the user what to do next. Show it under the message. If it is `null`, there is nothing for the user to do.
3. **Codes never change their name.** A code can be added, and one can be retired, but an existing code is never renamed. An unknown code should fall back to showing the message and the advice.

4. Use `retryable` for the retry button. If it is `true`, trying the same thing again can work. If it is `false`, the user must change something first.
5. Always keep the reference. Every coded error carries a `reference`. Show it wherever the user is told to contact support. Our engineers can find the exact request from it.

2. What an Error Response Looks Like

Every error comes back in the same wrapper. The details of a coded error sit inside `error.field_errors`.

```
HTTP 413

{
  "success": false,
  "message": "big.pdf is 3.0 MB and the limit is 1 MB.",
  "error": {
    "field_errors": {
      "error": "big.pdf is 3.0 MB and the limit is 1 MB.",
      "reason_code": "FILE_TOO_LARGE",
      "error_class": "USER",
      "remediation": "Split the file, remove blank pages or scan at a lower resolution, then upload again.",
      "retryable": false,
      "params": {
        "file_name": "big.pdf",
        "dimension": "bytes",
        "actual": 3145736,
        "limit": 1048576
      },
      "reference": "the same value as the X-Request-ID response header"
    }
  }
}
```

Field	What it is
<code>success</code>	Always <code>false</code> on an error. There is no <code>data</code> key on an error.
<code>message</code>	The one sentence to show the user. The same text as <code>field_errors.error</code> .
<code>reason_code</code>	The fixed name of the problem. This is what the frontend decides on.
<code>error_class</code>	Who or what caused it. See the table below.
<code>remediation</code>	What the user should do next, or <code>null</code> .
<code>retryable</code>	<code>true</code> if the same action can simply be tried again.

Field	What it is
<code>params</code>	The facts behind the message, as plain values the frontend can use. May be empty. It never holds an internal id, and it holds an email address for only three codes.
<code>reference</code>	The request's id, made by the server. It is also sent as the <code>X-Request-ID</code> response header on every response. An id sent by the client is never used.
<code>code</code>	An older key kept for two codes only: <code>"insufficient_credits"</code> and <code>"ai_feature_not_available"</code> . New code should read <code>reason_code</code> .

The five error classes

error_class	Meaning in plain words
<code>USER</code>	The user can fix it: a wrong file, a missing name, no credits.
<code>VALIDATION</code>	Something needed is not set up yet, such as an assignment with no marking guide.
<code>PROVIDER</code>	An outside service, such as the AI service, failed. Usually worth trying again.
<code>MODEL</code>	The AI answered but its answer could not be used. No user-facing code uses this today.
<code>SYSTEM</code>	A fault on our side. Used for a batch item that failed with no reason code.

A successful response, for comparison

```
{ "success": true, "message": "Request Successful", "data": { ... } }
```

3. Errors Inside a Batch

Some actions handle many things at once: a class set of papers, a class list, several teachers. One bad item must not sink the rest. So the request as a whole succeeds, and each item reports its own result with its own reason code.

3.1 Batches that answer straight away

These return `HTTP 200` with a list. A problem item is an entry in that list carrying the same coded fields as section 2, plus that route's own fields.

Action	Route	What comes back in <code>data</code>
Import a class list	<code>POST</code> <code>course/{id}/bulk-add-students</code>	<code>total_processed</code> , <code>success_count</code> , <code>failure_count</code> , <code>skipped_count</code> , <code>results[]</code> . Each result has <code>row</code> , <code>name</code> and <code>status</code> : <code>enrolled</code> , <code>invited</code> , <code>skipped</code> or <code>failed</code> .
Add teachers to a licence	<code>POST</code> <code>license-subscriptions/{id}/add_teachers</code> <code>body</code> <code>{ "teacher_emails": [...]</code>	<code>successful</code> , <code>failed</code> , <code>added[]</code> , <code>skipped[]</code> , <code>errors[]</code> . Entries carry <code>teacher_email</code> and <code>status</code> .
Remove teachers from a licence	<code>POST</code> <code>license-subscriptions/{id}/remove_teachers</code> <code>body</code> <code>{ "teacher_ids": [...]</code>	<code>successful</code> , <code>failed</code> , <code>removed[]</code> , <code>errors[]</code> . Entries carry <code>teacher_id</code> and <code>status</code> .
Release all grades	<code>POST</code> <code>assignments/{id}/publish-all-grades</code>	<code>message</code> , <code>total_graded</code> , <code>ungraded_count</code> , <code>skipped[]</code> . Each skipped entry has <code>submission_id</code> , <code>student_id</code> and <code>status</code> : <code>"skipped"</code> .

A row that was added has no coded fields:

```
{ "row": 1, "name": "Ann One", "status": "enrolled", "type": "direct_add" }
```

A row that was not added has them all. Note that `error` and `message` hold the same text:

```
{
  "row": 4,
  "name": "Ben Two",
  "status": "failed",
  "error": "Row 4: \"ben@\" isn't a valid email address.",
  "reason_code": "ROW_EMAIL_INVALID",
  "error_class": "USER",
  "message": "Row 4: \"ben@\" isn't a valid email address.",
  "remediation": "Correct the email and import the row again.",
  "retryable": false,
  "params": { "row": 4, "email": "ben@" },
  "reference": "the X-Request-ID of this request"
}
```

3.2 Batches that run in the background

Uploading a set of papers and grading a whole assignment take time, so they answer `HTTP 202` at once with a `session_id`. The frontend then asks for progress.

Step	Route
Start: upload a set of papers	<code>POST submissions/{assignment_id}/batch-upload</code>
Start: upload assignments	<code>POST assignments/upload-async</code>
Start: grade every submission	<code>POST assignments/{id}/grade-all</code>
Ask for progress and results	<code>GET tasks/session-results/{session_id}</code>

The start response is `{ session_id, message, tasks: [{ file_name, task_id, item_id }] }`. (`grade-all` does not include `item_id` there.)

What the progress response holds

Field	Meaning
<code>progress</code> , <code>percent</code> , <code>is_complete</code>	How far the batch has got. <code>progress</code> is text such as <code>"12 / 30"</code> .
<code>success_count</code> , <code>failure_count</code> , <code>cancelled_count</code> , <code>pending_count</code>	How many items are in each state.
<code>success_list</code> , <code>failure_list</code> , <code>cancelled_list</code> , <code>pending_list</code>	The items themselves. All four are always present, in the order the items were sent. <code>pending_list</code> holds items that are waiting and items that are running.
<code>failure_codes</code>	A count of failures by code, for example <code>{"FILE_UNREADABLE": 2, "MISSING_STUDENT_NAME": 1}</code> . Failures with no code are counted under <code>"UNCLASSIFIED"</code> .
<code>stopped_at_item</code>	The position of the first item that ran out of credits, or <code>null</code> .
<code>resumable</code>	<code>true</code> if at least one failed item is marked retryable.

One item in those lists

```
{
  "item_id": "id of this item, used to retry it",
  "item_index": 7,
  "task_id": "background task id, or null if the item was refused before it started",
  "file_name": "p07.pdf",
  "submission_id": null,
  "status": "FAILURE",
  "reason_code": "MISSING_STUDENT_NAME",
  "error_class": "USER",
  "message": "We couldn't match p07.pdf to a student: no name was found on the paper.",
  "error": "the same text as message",
  "remediation": "Choose the student this paper belongs to, or add them to the course.",
  "retryable": false,
  "retry_count": 0,
  "reference": "the X-Request-ID of the request that started the batch",
  "replaced_existing": null,
  "context": { "resource_type": "...", "resource_id": "...", "action": "...", "additional_ids": {} }
}
```

- **status** is one of `PENDING`, `STARTED`, `SUCCESS`, `FAILURE`, `CANCELLED`.
- **message** is filled only when the item failed.
- **replaced_existing** is `true` or `false` on a successful upload, and tells you the paper replaced an earlier one for the same student.
- There is no **params** on these items. The facts are already written into **message**.
- After a retry, **reference** becomes the id of the retry request and **retry_count** goes up by one.

4. Retrying a Failed Item

Action	Route	Answer
Retry one item	<code>POST</code> <code>tasks/session/{session_id}/items/{item_id}/retry</code> no body	<code>202</code> with the item, now <code>PENDING</code> , <code>retry_count</code> one higher, a new <code>task_id</code> .
Retry every failed item	<code>POST</code> <code>tasks/session/{session_id}/retry-failed</code> optional body <code>{"reason_codes": ["PROVIDER_FAILURE"]}</code>	<code>202</code> with <code>{ "retried": [item_id, ...], "skipped": [{ "item_id", "reason_code" }] }</code> .

- **What can be retried in place:** a failed *grading* item whose code is retryable. In practice that is `PROVIDER_FAILURE` and `INSUFFICIENT_CREDITS_MID_BATCH`.

- **Upload items cannot be retried in place**, because the uploaded file is not kept. The answer is `409 NOT_RETRYABLE` with `params: {"resolution": "replace_file"}`. The user must upload the file again.
- **Other answers:** `404` if the session or item is not this teacher's; `402 INSUFFICIENT_CREDITS` if the wallet is empty; `409 NOT_RETRYABLE` if the item did not fail or its code is not retryable.
- **After a credit top-up**, call `retry-failed` with `{"reason_codes": ["INSUFFICIENT_CREDITS_MID_BATCH"]}`. Finished items are left alone.

5. The Codes, One by One

Each card gives the HTTP status used when the code answers a whole request, who can fix it, whether it is retryable, the exact message and advice, the `params` it may carry, and where it is returned. Words in `{braces}` are filled in by the backend. When a code appears on an item inside a batch, the request itself still answers 200 or 202.

Files and uploads

A file could not be accepted. The teacher fixes the file and uploads it again.

FILE_TOO_LARGE

HTTP 413

USER

Not retryable

Message **`{file_name}` is `{actual}` and the limit is `{limit}`.**

What to do Split the file, remove blank pages or scan at a lower resolution, then upload again.

Params `actual`, `dimension`, `file_name`, `limit`

Returned by Whole request: `POST submissions/{assignment_id}/upload`, `.../upload-async`, `POST course/{id}/bulk-add-students`. Per item: `submissions/{assignment_id}/batch-upload`, `assignments/upload-async`.

Notes `dimension` is `"bytes"`, `"pages"` or `"pixels"`. `limit` is always a whole number in that unit. `actual` is a whole number, and is left out when the true size is not known. The friendly sizes (`"3.0 MB"`) are only in the message.

FILE_TYPE_UNSUPPORTED

HTTP 415

USER

Not retryable

Message	{file_name} is a {detected_type} file, which isn't supported. Accepted types: {accepted_types}.
What to do	Save or export the file as a PDF or an image, and upload it again.
Params	accepted_types , detected_type , file_name
Returned by	Whole request: POST submissions/{assignment_id}/upload . Per item: the two async batch upload routes.
Notes	detected_type is the file extension in capitals ("TXT"), or the declared type ("application/zip"), or "unknown" . Accepted types are PDF, JPEG, PNG, GIF and WebP.

FILE_NOT_A_PDF

HTTP 422

USER

Not retryable

Message	{file_name} is not a PDF. If it is a photo or scan, upload it as an image instead.
What to do	Upload the photo or scan as an image (JPEG, PNG, GIF or WebP).
Params	file_name
Returned by	Whole request: POST submissions/{assignment_id}/upload . Per item: the two async batch upload routes.
Notes	The file is named or declared as a PDF but its contents are not one.

FILE_UNREADABLE

HTTP 422

USER

Not retryable

Message	We couldn't read {file_name}. It may be damaged, password-protected or incomplete.
What to do	Re-export or re-scan the file and upload it again.
Params	file_name
Returned by	Whole request: POST submissions/{assignment_id}/upload . Per item: the two async batch upload routes.
Notes	Damaged, password-protected or cut-off files.

SUBMISSION_EMPTY

HTTP 422

USER

Not retryable

Message **{file_name} has no student answers to grade.**

What to do Check that the right file was uploaded, then upload it again or mark the submission as missing.

Params `file_name`

Returned by Whole request: `POST submissions/{assignment_id}/upload` ; `PATCH submissions/{id}` and `POST submissions/{id}/update-async` when the typed answer is blank. Per item: `batch-upload` .

Notes For typed answers `file_name` is the text "The submitted text".

Matching a paper to a student

The paper was read, but the system could not tie it to one student in the course.

MISSING_STUDENT_NAME

HTTP 422

USER

Not retryable

Message **We couldn't match {file_name} to a student: {name_state}.**

What to do Choose the student this paper belongs to, or add them to the course.

Params `file_name` , `name_state`

Returned by Per item only: `POST submissions/{assignment_id}/batch-upload` .

Notes `name_state` is one of three sentences: *no name was found on the paper; the name "..."* matches more than one student; the name "... doesn't match anyone on the roster.

STUDENT_NOT_ON_ROSTER

HTTP 422

USER

Not retryable

Message **{file_name} belongs to {student_display}, who isn't enrolled in this course.**

What to do Add the student to the course roster, then retry this paper.

Params `file_name` , `student_display`

Returned by Per item only: `POST submissions/{assignment_id}/batch-upload` .

Notes `student_display` is the student's first and last name.

DUPLICATE_SUBMISSION

HTTP 409

USER

Not retryable

Message **`{file_name}` is a second submission for `{student_display}`. The existing submission (`{existing_ref}`) was kept.**

What to do Choose which submission to keep.

Params `existing_ref` , `file_name` , `student_display`

Returned by Reserved. No endpoint returns it today.

Notes When a paper replaces an earlier one for the same student, the item succeeds and carries `replaced_existing: true` . Handle that flag, not this code.

Grading, credits and plan

Grading could not start or could not finish.

RUBRIC_MISSING

HTTP 409

VALIDATION

Not retryable

Message **This assignment has no rubric to grade against, so grading hasn't started. No credits were used.**

What to do Add questions and a rubric to the assignment, then grade again.

Params none

Returned by Whole request: `POST submissions/{id}/grade` , `.../grade-async` , `.../schedule-grade-async` , `POST assignments/{id}/grade-all` , `POST assignments/{id}/schedule_grade_all_submission` . Per item: when a scheduled or automatic grading run fires.

Notes Nothing is charged.

PROVIDER_FAILURE

HTTP 503

PROVIDER

Retryable

Message	The grading service couldn't finish this item. {credit_clause}
What to do	Try again in a few minutes. If it keeps happening, contact support and quote the reference.
Params	<code>credit_clause</code>
Returned by	Whole request: <code>POST submissions/{id}/grade</code> , <code>PATCH submissions/{id}</code> , <code>POST submissions/{assignment_id}/upload</code> . Per item: any batch.
Notes	Sent with the header <code>Retry-After: 30</code> . <code>credit_clause</code> is either "No credits were charged." or "The credits were refunded." A failed grading item with this code can be retried in place.

INSUFFICIENT_CREDITS

HTTP 402

USER

Not retryable

Message	There aren't enough AI credits available for this. The credit wallet needs to be topped up before it can run.
What to do	Top up credits, or ask your school administrator, then try again.
Params	none
Returned by	Whole request: every AI route, and both retry routes, when the wallet is empty. Per item: the first item of a batch, when nothing had run yet.
Notes	Also carries the older key <code>code: "insufficient_credits"</code> . Keep reading <code>reason_code</code> .

INSUFFICIENT_CREDITS_MID_BATCH

HTTP 402

USER

Retryable

Message	Credits ran out after <code>{completed}</code> of <code>{total}</code> items. The finished items are saved.
What to do	Top up credits, then resume the remaining items. Unfinished uploads need their files uploaded again.
Params	<code>completed</code> , <code>total</code>
Returned by	Per item only, in any batch.
Notes	When no item had finished, the message is "Credits ran out before any of the <code>{total}</code> items were finished." The session reports <code>stopped_at_item</code> . After a top-up, grading items can be resumed with <code>retry-failed</code> ; upload items must be uploaded again.

AI_FEATURE_NOT_AVAILABLE

HTTP 403

USER

Not retryable

Message	This feature isn't included in your plan.
What to do	Upgrade your plan, or ask your school administrator to enable it.
Params	none
Returned by	Whole request: AI routes the user's plan does not include. Can also appear on batch items.
Notes	Also carries the older key <code>code: "ai_feature_not_available"</code> .

NOT_RETRYABLE

HTTP 409

USER

Not retryable

Message	<code>{why}</code>
What to do	Fix what its failure message describes first, then try again.
Params	<code>resolution</code> , <code>why</code>
Returned by	Whole request: <code>POST tasks/session/{session_id}/items/{item_id}/retry</code> only.
Notes	For an upload item the message is "This upload can't be retried as it is, because its file isn't kept. Upload the file again." and <code>params</code> is <code>{"resolution": "replace_file"}</code> . For anything else the message is "This item can't be retried as it is." and <code>params</code> is empty.

SUBMISSION_NOT_GRADED

HTTP 400

USER

Not retryable

Message	This submission hasn't been graded yet, so it can't be published.
What to do	Grade it first, then publish.
Params	none
Returned by	Whole request: <code>POST submissions/{id}/publish</code> . Per item: the <code>skipped</code> list of <code>POST assignments/{id}/publish-all-grades</code> .

Sign-in locks

Too many wrong attempts. The user has to wait. These four have a slightly different body, described in section 6.

ACCOUNT_LOCKED

HTTP 401

USER

Retryable

Message	Too many failed login attempts. Please try again later.
What to do	Wait a few minutes and try again, or reset your password.
Params	none
Returned by	<code>POST auth/login</code>
Notes	Five wrong passwords lock the account for 15 minutes. The body also has <code>detail</code> and the older key <code>code</code> : <code>"account_locked"</code> (lower case).

RESET_LOCKED

HTTP 429

USER

Retryable

Message	Password reset is paused on this account for a while, because the code was entered incorrectly too many times.
What to do	Wait until the time shown, then request a new code. Your password has not been changed.
Params	none
Returned by	<code>POST auth/reset-password</code>
Notes	The body also has <code>code</code> , <code>message</code> , <code>locked_until</code> (a UTC time) and <code>retry_after_seconds</code> . The <code>Retry-After</code> header carries the same number of seconds.

VERIFY_LOCKED

HTTP 429

USER

Retryable

Message	Too many incorrect codes for this email address.
What to do	Wait, then request a new verification email.
Params	none
Returned by	POST auth/verify
Notes	The body also has detail and code . A Retry-After header is sent, and the text ends with "Expected available in N seconds."

REGISTRATION_PAUSED

HTTP 429

USER

Retryable

Message	Student registration is paused for a short while because of too many invalid activation codes. Please try again later; if your code has expired by then, ask for a new one.
What to do	Try again in a few minutes. If your code has expired, ask your teacher for a new one.
Params	none
Returned by	POST auth/register/student
Notes	The body also has detail and code . A Retry-After header is sent (1 to 3600 seconds).

Roster import: the whole request

The class list could not be processed at all. Nothing was added.

ROSTER_NO_INPUT

HTTP 400

USER

Not retryable

Message	Upload a roster file or paste your student list.
What to do	Choose a CSV file, or paste rows copied from your spreadsheet.
Params	none
Returned by	Whole request: POST course/{id}/bulk-add-students

ROSTER_EMPTY

HTTP 400

USER

Not retryable

Message	This roster has no student rows.
What to do	Check that the file has one student per row, then try again.
Params	none
Returned by	Whole request: <code>POST course/{id}/bulk-add-students</code>

ROSTER_FILE_UNREADABLE

HTTP 400

USER

Not retryable

Message	{file_name} isn't readable as text.
What to do	Export your roster as a CSV file (UTF-8) and try again.
Params	<code>file_name</code>
Returned by	Whole request: <code>POST course/{id}/bulk-add-students</code>

ROSTER_TOO_MANY_ROWS

HTTP 400

USER

Not retryable

Message	This roster has {row_count} rows. Upload at most {max_rows} rows at a time.
What to do	Split the roster into smaller files.
Params	<code>max_rows</code> , <code>row_count</code>
Returned by	Whole request: <code>POST course/{id}/bulk-add-students</code>
Notes	The limit is 2,000 rows. A roster file over 2 MB answers FILE_TOO_LARGE instead.

Roster import: one row

The import ran, and this one row was skipped or failed. Other rows are unaffected.

ROW_NAME_MISSING

HTTP 422

USER

Not retryable

Message **Row {row}: a first and a last name are required.**

What to do Add the missing name and import the row again.

Params `row`

Returned by Per row: `results[]` of `POST course/{id}/bulk-add-students`, with `status: "failed"`.

ROW_NAME_INVALID

HTTP 422

USER

Not retryable

Message **Row {row}: each name needs between 2 and 150 characters.**

What to do Correct the name and import the row again.

Params `row`

Returned by Per row: `results[]` of `POST course/{id}/bulk-add-students`, with `status: "failed"`.

ROW_EMAIL_INVALID

HTTP 422

USER

Not retryable

Message **Row {row}: "{email}" isn't a valid email address.**

What to do Correct the email and import the row again.

Params `email`, `row`

Returned by Per row: `results[]` of `POST course/{id}/bulk-add-students`, with `status: "failed"`.

Notes One of only three codes whose `params` may hold an email address.

ROW_NAME_CLASH

HTTP 422

USER

Not retryable

Message	Row {row}: a student named {student_display} is already in this course.
What to do	Add an email address to tell the two students apart.
Params	<code>row</code> , <code>student_display</code>
Returned by	Per row: <code>results[]</code> of <code>POST course/{id}/bulk-add-students</code> , with <code>status: "failed"</code> .
Notes	Has a second approved remediation, used when an email would not help: "Two students in one course can't have exactly the same name, because papers are matched to students by name. Add a middle name or initial to tell them apart." Always show the <code>remediation</code> the response gives.

ROW_STAFF_EMAIL

HTTP 422

USER

Not retryable

Message	Row {row}: this email can't be added as a student.
What to do	Use the student's own email address.
Params	<code>row</code>
Returned by	Per row: <code>results[]</code> of <code>POST course/{id}/bulk-add-students</code> , with <code>status: "failed"</code> .
Notes	Also returned when a row supplies a placeholder student address ending in <code>@student.local</code> . Clients should never send such an address.

ROW_OTHER_SCHOOL

HTTP 422

USER

Not retryable

Message	Row {row}: this account can't be added to this school. If you believe this is a mistake, contact your school administrator.
What to do	None. The response sends <code>null</code> . Nothing for the user to do.
Params	<code>row</code>
Returned by	Per row: <code>results[]</code> of <code>POST course/{id}/bulk-add-students</code> , with <code>status: "failed"</code> .

ROW_ALREADY_ENROLLED

HTTP 422

USER

Not retryable

Message	Row {row}: {student_display} is already in this course.
What to do	None. The response sends <code>null</code> . Nothing for the user to do.
Params	<code>row</code> , <code>student_display</code>
Returned by	Per row: <code>results[]</code> of <code>POST course/{id}/bulk-add-students</code> , with <code>status: "skipped"</code> .

ROW_DUPLICATE

HTTP 422

USER

Not retryable

Message	Row {row} repeats row {first_row}.
What to do	None. The response sends <code>null</code> . Nothing for the user to do.
Params	<code>first_row</code> , <code>row</code>
Returned by	Per row: <code>results[]</code> of <code>POST course/{id}/bulk-add-students</code> , with <code>status: "skipped"</code> .

ROW_ACCOUNT_DISABLED

HTTP 422

USER

Not retryable

Message	Row {row}: this student's account is disabled.
What to do	Contact support if they should have access.
Params	<code>row</code>
Returned by	Per row: <code>results[]</code> of <code>POST course/{id}/bulk-add-students</code> , with <code>status: "skipped"</code> .

ROW_FAILED

HTTP 422

USER

Retryable

Message

Row {row}: this student couldn't be added.

What to do

Check the row and try again. If it keeps failing, contact support and quote the reference.

Params

row

Returned by

Per row: `results[]` of `POST course/{id}/bulk-add-students`, with `status: "failed"`.

School licence: the whole request

Adding or removing teachers could not start.

TEACHER_LIST_EMPTY

HTTP 400

USER

Not retryable

Message

Add at least one teacher.

What to do

Enter the teachers' email addresses.

Params

none

Returned by

Whole request: `add_teachers` and `remove_teachers`

Notes

On `remove_teachers` the remediation is "Choose the teachers to remove."

LICENCE_INACTIVE

HTTP 400

USER

Not retryable

Message

This licence isn't active, so teachers can't be added to it.

What to do

Renew the licence, or contact us.

Params

none

Returned by

Whole request: `add_teachers`

LICENCE_SEATS_EXCEEDED

HTTP 400

USER

Not retryable

Message	Your licence has {availability} ({in_use} of {max_seats} in use).
What to do	Add fewer teachers, remove a teacher, or ask us to add seats.
Params	<code>adding</code> , <code>availability</code> , <code>in_use</code> , <code>max_seats</code> , <code>remaining</code>
Returned by	Whole request: <code>add_teachers</code>
Notes	<code>params</code> holds four whole numbers: <code>remaining</code> , <code>adding</code> , <code>in_use</code> , <code>max_seats</code> . The words in the message ("2 seats left, but you're adding 5 teachers" or "no seats left") are not in <code>params</code> .

School licence: one teacher

The request ran, and this one teacher was not added or removed.

TEACHER_EMAIL_NOT_BUSINESS

HTTP 422

USER

Not retryable

Message	{email} isn't a school or work email address.
What to do	Use the teacher's school or work email.
Params	<code>email</code>
Returned by	Per teacher: <code>errors[]</code> of <code>add_teachers</code> , with <code>status: "failed"</code> .
Notes	<code>params.email</code> is the address that was typed.

TEACHER_EMAIL_OTHER_ROLE

HTTP 422

USER

Not retryable

Message	This email can't be added as a teacher.
What to do	Use the teacher's own account email.
Params	<code>none</code>
Returned by	Per teacher: <code>errors[]</code> of <code>add_teachers</code> , with <code>status: "failed"</code> .

TEACHER_CANNOT_JOIN_YET

HTTP 422

USER

Not retryable

Message	This teacher can't be added to your school yet. Please ask them to contact support.
What to do	Ask the teacher to contact support.
Params	none
Returned by	Per teacher: <code>errors[]</code> of <code>add_teachers</code> , with <code>status: "failed"</code> .
Notes	New on 5 October 2026. It replaces <code>TEACHER_HAS_INDIVIDUAL_SUBSCRIPTION</code> , which no longer exists. The new code has no <code>email</code> in <code>params</code> . Do not show any wording about the teacher's own subscription.

TEACHER_IN_OTHER_SCHOOL

HTTP 422

USER

Not retryable

Message	This teacher already belongs to another school.
What to do	Contact support if the teacher has moved schools.
Params	none
Returned by	Per teacher: <code>errors[]</code> of <code>add_teachers</code> , with <code>status: "failed"</code> .

TEACHER_ALREADY_ON_LICENCE

HTTP 422

USER

Not retryable

Message	{email} is already on this licence.
What to do	None. The response sends <code>null</code> . Nothing for the user to do.
Params	<code>email</code>
Returned by	Per teacher: <code>skipped[]</code> of <code>add_teachers</code> , with <code>status: "skipped"</code> .
Notes	Not a failure. The teacher is already there.

TEACHER_ADD_FAILED

HTTP 422

USER

Retryable

Message	We couldn't add this teacher.
What to do	Try again. If it keeps failing, contact support and quote the reference.
Params	none
Returned by	Per teacher: <code>errors[]</code> of <code>add_teachers</code> , with <code>status: "failed"</code> .

TEACHER_NOT_ON_LICENCE

HTTP 422

USER

Not retryable

Message	This teacher isn't an active teacher on this licence.
What to do	None. The response sends <code>null</code> . Nothing for the user to do.
Params	none
Returned by	Per teacher: <code>errors[]</code> of <code>remove_teachers</code> .
Notes	Given alike for an unknown teacher, a teacher in another school and a teacher not on this licence, so the answer reveals nothing about other schools.

TEACHER_REMOVE_FAILED

HTTP 422

USER

Retryable

Message	We couldn't remove this teacher.
What to do	Try again. If it keeps failing, contact support and quote the reference.
Params	none
Returned by	Per teacher: <code>errors[]</code> of <code>remove_teachers</code> .

6. Sign-in Locks: a Slightly Different Body

The four lock codes are added to responses that existed before reason codes did. Their bodies therefore differ from section 2 in three ways:

- **There is no `error` key inside `field_errors`.** The text is in `detail`, or in `message` for `RESET_LOCKED`. The top-level `message` still holds the sentence to show.
- **There is an extra `code` key.** It equals the reason code, except for sign-in, where it is the older lower-case `"account_locked"`.

- `params` is always empty, and `retryable` is always `true` : the same action works once the wait is over.

HTTP 429 Retry-After: 1740

```
{
  "success": false,
  "message": "Too many incorrect codes for this email address. Expected available in 1740 seconds.",
  "error": {
    "field_errors": {
      "detail": "Too many incorrect codes for this email address. Expected available in 1740 seconds.",
      "code": "VERIFY_LOCKED",
      "reason_code": "VERIFY_LOCKED",
      "error_class": "USER",
      "remediation": "Wait, then request a new verification email.",
      "retryable": true,
      "params": {},
      "reference": "the X-Request-ID of this request"
    }
  }
}
```

Code	Route	Status	Extra fields	How long to wait
ACCOUNT_LOCKED	POST auth/login	401	detail, code: "account_locked"	15 minutes. No <code>Retry-After</code> header.
RESET_LOCKED	POST auth/reset-password	429	code, message, locked_until, retry_after_seconds	<code>retry_after_seconds</code> , also in the <code>Retry-After</code> header.
VERIFY_LOCKED	POST auth/verify	429	detail, code	The <code>Retry-After</code> header.
REGISTRATION_PAUSED	POST auth/register/student	429	detail, code	The <code>Retry-After</code> header.

7. Errors That Have No Reason Code

Not every error is coded. The frontend should handle these by showing the top-level `message`.

- **Ordinary form errors.** A missing or badly formed field answers `400` with the field names inside `field_errors` and no `reason_code`.
- **Not found and not allowed.** `404` and `403` answers carry a plain `detail`.
- **Too many requests from one address.** The ordinary rate limit answers `429` with no `code` and no `reason_code`. Only the four locks in section 6 are coded.

- **A batch item that failed for an unknown reason.** It has `reason_code: null`, `error_class: "SYSTEM"`, `remediation: null`, `retryable: false`, and a safe general message. It is counted under `"UNCLASSIFIED"` in `failure_codes`.
- **Uploading assignment files with `POST assignments/upload`**. Failed files come back as `{ file_name, error }` with text only. A mixed result answers `207`; all succeeded answers `201` with a plain list; all failed answers `400`.
- **Adding one student** with `POST course/{id}/students` or `POST course/{id}/direct-add-student`. Refusals carry text only.
- **Checking one background task** with `GET tasks/status/{task_id}`. It reports the state but no reason code. Use the session results for codes.
- **Unexpected server faults** answer `500` with "An unexpected error occurred. Please try again."

8. Codes Kept Only in the Activity Record

These 19 codes are written into the permanent activity record so that support staff and school administrators can see what happened. **They are never sent as `reason_code` in a response.** Many describe failed sign-ins, where telling the visitor the exact reason would help someone guessing passwords. The right-hand column shows what the user receives instead.

Code	Recorded when	What the user receives
<code>WRONG_PASSWORD</code>	Sign-in with a wrong password; change-password with a wrong current password	401 on sign-in; 400 "Incorrect current password. Please try again." on change-password
<code>INVALID_CREDENTIALS</code>	Sign-in refused for a reason other than a wrong password, such as an unknown address	401, the same answer as a wrong password
<code>ACCOUNT_DEACTIVATED</code>	Google sign-in to a deactivated account	401 "This account has been deactivated. Please contact support."
<code>INVALID_CODE</code>	A wrong code on verify, reset-password, change-password, student registration or school-admin registration	400 with that screen's own text, for example "Invalid email or token."
<code>CODE_EXPIRED</code>	An expired code on the same five screens	400 with that screen's own text. Student registration answers 200 with a renewal link
<code>CODE_MISSING</code>	Verify called without an email or code	400 "Email and Token are required."

Code	Recorded when	What the user receives
CODE_NOT_REQUESTED	Change-password with a code when none was requested	400 "No password change code has been requested for this account. Request one and try again."
REFRESH_TOKEN_MISSING	Sign-out without a refresh token	400 "Refresh token is required."
REFRESH_TOKEN_INVALID	Sign-out with a bad or expired refresh token	400 "Invalid or expired token"
SESSION_REVOKE_FAILED	Sign-out failed on the server	500
GOOGLE_CODE_MISSING	Google sign-in without an authorisation code	400 "Authorization code is required"
GOOGLE_EXCHANGE_FAILED	Google did not accept the authorisation code	400 "Google sign-in failed. Please try again."
GOOGLE_TOKEN_INVALID	Google's identity token was missing or failed its check	400 "Google did not return an ID token" or "Invalid Google token signature"
GOOGLE_EMAIL_UNVERIFIED	Google has not verified the user's email	400 "Google has not verified your email"
GOOGLE_SIGN_IN_REFUSED	Any other refused Google sign-in	400 with the form's own error
INVALID_REQUEST	A badly formed request to a sign-in or registration screen	400 with ordinary field errors
SERVER_ERROR	A request from a signed-out visitor that crashed	500 "An unexpected error occurred. Please try again."
FAILED_AUTH_CAPPED	A summary entry written when many failed sign-ins on one account are grouped	Nothing. It has no effect on any response
COURSE_NOT_REACHABLE	Background grading stopped because the teacher can no longer reach the course	"This course wasn't found." on the item; a plain 404 on a direct request

How QA can check them

Read the activity record after the action. Every user-facing code is recorded there as well, so the same check works for all 63.

Who	Route	Sees
Platform superadmin	GET super-admin/audit/events	Every school

Who	Route	Sees
School administrator	<code>GET school-admin/audit/events</code>	Their own school only

Filters: `reason_code` (exact), `outcome`, `action`, `actor_id`, `actor_role`, `route`, `time_from`, `time_to`, `page`, `page_size` (at most 100). The superadmin route also takes `school_id`. Each entry's `trace_id` equals the `reference` the user was shown, which is how a support request is matched to a record.

9. Limits Worth Knowing for Testing

Limit	Value	Code when exceeded
Size of one uploaded file	50 MB	<code>FILE_TOO_LARGE</code> , <code>dimension: "bytes"</code>
Pages in one PDF	300	<code>FILE_TOO_LARGE</code> , <code>dimension: "pages"</code>
Size of one image	50 million pixels	<code>FILE_TOO_LARGE</code> , <code>dimension: "pixels"</code>
Image size after compression	4 MB	<code>FILE_TOO_LARGE</code> , <code>dimension: "bytes"</code>
Class list file	2 MB	<code>FILE_TOO_LARGE</code> , <code>dimension: "bytes"</code>
Rows in one class list	2,000	<code>ROSTER_TOO_MANY_ROWS</code>
Wrong passwords before a lock	5, then 15 minutes	<code>ACCOUNT_LOCKED</code>
Wrong reset codes before a lock	5, then 30 minutes	<code>RESET_LOCKED</code>
Wrong verification codes before a lock	5, then 30 minutes	<code>VERIFY_LOCKED</code>

A short checklist for each code

1. The HTTP status matches the card (for a whole-request error).
2. `reason_code` is exactly the name on the card.
3. `message` matches the card with the blanks filled in, and contains no technical text.
4. `remediation` matches the card, or is `null` where the card says none.
5. `retryable` matches the card.
6. `reference` equals the `X-Request-ID` response header, even when the request sent its own id.
7. The matching entry appears in the activity record with the same code and the same `trace_id`.

10. Quick Index of All User-facing Codes

Code	Group	HTTP	Retryable
FILE_TOO_LARGE	Files and uploads	413	No
FILE_TYPE_UNSUPPORTED	Files and uploads	415	No
FILE_NOT_A_PDF	Files and uploads	422	No
FILE_UNREADABLE	Files and uploads	422	No
SUBMISSION_EMPTY	Files and uploads	422	No
MISSING_STUDENT_NAME	Matching a paper to a student	422	No
STUDENT_NOT_ON_ROSTER	Matching a paper to a student	422	No
DUPLICATE_SUBMISSION	Matching a paper to a student	409	No
RUBRIC_MISSING	Grading, credits and plan	409	No
PROVIDER_FAILURE	Grading, credits and plan	503	Yes
INSUFFICIENT_CREDITS	Grading, credits and plan	402	No
INSUFFICIENT_CREDITS_MID_BATCH	Grading, credits and plan	402	Yes
AI_FEATURE_NOT_AVAILABLE	Grading, credits and plan	403	No
NOT_RETRYABLE	Grading, credits and plan	409	No
SUBMISSION_NOT_GRADED	Grading, credits and plan	400	No
ACCOUNT_LOCKED	Sign-in locks	401	Yes
RESET_LOCKED	Sign-in locks	429	Yes
VERIFY_LOCKED	Sign-in locks	429	Yes
REGISTRATION_PAUSED	Sign-in locks	429	Yes
ROSTER_NO_INPUT	Roster import: the whole request	400	No
ROSTER_EMPTY	Roster import: the whole request	400	No
ROSTER_FILE_UNREADABLE	Roster import: the whole request	400	No
ROSTER_TOO_MANY_ROWS	Roster import: the whole request	400	No

Code	Group	HTTP	Retryable
ROW_NAME_MISSING	Roster import: one row	422	No
ROW_NAME_INVALID	Roster import: one row	422	No
ROW_EMAIL_INVALID	Roster import: one row	422	No
ROW_NAME_CLASH	Roster import: one row	422	No
ROW_STAFF_EMAIL	Roster import: one row	422	No
ROW_OTHER_SCHOOL	Roster import: one row	422	No
ROW_ALREADY_ENROLLED	Roster import: one row	422	No
ROW_DUPLICATE	Roster import: one row	422	No
ROW_ACCOUNT_DISABLED	Roster import: one row	422	No
ROW_FAILED	Roster import: one row	422	Yes
TEACHER_LIST_EMPTY	School licence: the whole request	400	No
LICENCE_INACTIVE	School licence: the whole request	400	No
LICENCE_SEATS_EXCEEDED	School licence: the whole request	400	No
TEACHER_EMAIL_NOT_BUSINESS	School licence: one teacher	422	No
TEACHER_EMAIL_OTHER_ROLE	School licence: one teacher	422	No
TEACHER_CANNOT_JOIN_YET	School licence: one teacher	422	No
TEACHER_IN_OTHER_SCHOOL	School licence: one teacher	422	No
TEACHER_ALREADY_ON_LICENCE	School licence: one teacher	422	No
TEACHER_ADD_FAILED	School licence: one teacher	422	Yes
TEACHER_NOT_ON_LICENCE	School licence: one teacher	422	No
TEACHER_REMOVE_FAILED	School licence: one teacher	422	Yes

Prepared 5 October 2026 from the reason-code catalogue in the backend code on the staging environment.
Messages and advice are copied from that catalogue word for word.